1

Lesson: C Fundamentals

CSE 4107 : Structured Programming I

Shahriar Ivan

2

Writing a Simple Program

Your first C program: "Hello, world"

```
#include <stdio.h>
int main(void){
    printf("Hello, world\n");
    return 0;
}
```

- The program can be saved in a file named *hello.c*
- You can name it anything, but the .c extension is often required

3

Compiling and Linking

- Before a program can be executed, three steps are usually required
  - ***Preprocessing.*** The ***preprocessor*** obeys commands that begin with **#** (known as ***directives***)
  - ***Compiling.*** A ***compiler*** translates the program into machine code (***object code***)
  - ***Linking.*** A ***linker*** combines the object code produced by the compiler with any additional code needed (like, ***printf***) to yield a complete executable program
- The preprocessor is usually integrated with the compiler

In the command line: cc -o hello hello.c or gcc -o hello hello.c

7

General form of a Simple Program

Your first C program: "Hello, world"

```
#include <stdio.h>
int main(void){
    printf("Hello, world\n");
    return 0;
}
```

directives

```
int main(void){
    statements
}
```

8

General form of a Simple Program

- C uses **{** and **}** like other languages uses ***begin*** and ***end***
- Three key features:
  - Directives (***editing commands*** that modify program prior to compilation)
  - Functions (named ***blocks*** of executable codes)
  - Statements (***commands*** to be performed when the program is run)

9

General form of a Simple Program

Directives

- A preprocessor edits a C program before it is compiled
- Commands intended for the preprocessor are called directives
  - Example: `#include <stdio.h>`
- `<stdio.h>` is a ***header*** containing information about C's standard ***I/O*** library
- Begins with a # character
- One line long
- No semicolon or other special characters at the end

13

General form of a Simple Program

Printing Strings

- *printf* displays ***string literal*** - characters enclosed in double quotes
- *printf* doesn't automatically advance to the next output line
- To make *printf* advance one line, use *\n*
- The statement -> printf("Hello, world\n");
  - Can be replaced by two calls of *printf* :
  - printf("Hello,");
  - printf("world\n");

14

Comments

C89 Style

- A ***comment*** begins with **/*** and ends with ***/**

```
/*
Name: hello.c
Purpose: Prints Hello, world
Author: Mohammed Saidul Islam
*/
#include <stdio.h>
int main(void){   /* beginning */
        printf("Hello, world\n");   /* print function */
        return 0;
}
```

- Can appear in anywhere in the program
- Comments may extend over more than one line

15

Comments

C89 Style

```
/*
Name: hello.c
Purpose: Prints Hello, world
Author: Mohammed Saidul Islam

#include <stdio.h>
int main(void){   /* beginning */
        printf("Hello, world\n");   /* print function */
        return 0;
}
```

- Failure to end a comment can result in program statements being commented out, causing your program to be incorrect.

19

Variables and Assignment
Declarations

- Variables must be declared before they are used
- Variables can be declared one at a time:
  - `int height;`
  - `float profit;`
- Alternatively, several variables can be declared at the same time:
  - `int height, length, width, volume;`
  - `float profit, loss;`

20

Variables and Assignment
Assignment

- ***Assignment*** is the process of assigning a value to a variable.
  - `height = 8;`
  - `profit = 100.0;`
- Declare first, then assign

```
// Correct way
int height;
height = 8;
int length = 12, width = 10;
float profit = 100.0;
```

```
// Wrong way
height = 8;
int height;
// Not wrong, but be careful
int length, width = 10;
```

21

Variables and Assignment
Assignment

- It's best to append the letter f to a floating point constant
  - `float profit = 100.0f;`
- Assigning *int* to *float*, or vice versa is possible but not recommended
- Once assigned, a variable can be used to compute the value of another variable:
  - `int height = 10, length = 20, width = 5, volume;`
  - `volume = height * length * width; // volume is now 1000`
- The right side of an assignment can be a formula (or expression in C terminology) involving constants, variables, and operators

25

Program

Computing the dimensional weight of a box

- Shipping companies often charge extra for boxes that are large, but very light based on their volume instead of weight
- Usual method followed in the US:
  - Divide the volume of the box by 166 (the allowed number of cubic inches per pound) -> Dimensional weight
  - If the value exceeds its actual weight, shipping fee is based on dimensional weight
- We'll write a program, that computes the dimensional weight of a particular box:
  - Dimension: 12" x 10" x 8", Volume: 960 (cubic inches), Dimensional weight: 6

26

Program

Computing the dimensional weight of a box

- Division is represented by / in C:
  - `weight = volume / 166`
- In C, when one integer is divided by another, the answer is truncated
  - The volume will be 960 cubic inches
  - Dividing by 166 gives 5, instead of 5.783
- We'll write a program, that computes the dimensional weight of a particular box:
  - Dimension: 12" x 10" x 8", Volume: 960 (cubic inches), Dimensional weight: 6

27

Program - Initialization

Computing the dimensional weight of a box

- Some variables are automatically set to zero when a program begins to execute, but most are not
- A variable which doesn't have a default value and hasn't yet been assigned a value, is said to be ***uninitialized***

Note: Attempting to access the value of an uninitialized variable may yield an unpredictable result. With some compilers, worst-case behavior - even a program crash - may occur.

31

Defining Names for constants

- During preprocessing, the statement

  ```
  weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
  ```

  will become

  ```
  weight = (volume + 166 - 1) / 166;
  ```

- The value of a macro can be an expression:

  ```
  #define RECIPROCAL_OF_PI (1.0f / 3.14159f)
  ```

- If it contains operators, the expression should be enclosed in parentheses and using only uppercase letters in macro names is a common convention

32

Program

Converting from Fahrenheit to Celsius

- Given a temperature in Fahrenheit, convert it to Celsius
  - Formula: C = (F - Freezing Point) x Scale Factor
    = (F - 32) x (5/9)
- Sample program output:

  ```
  Enter Fahrenheit temperature: 212
  Celsius equivalent: 100.0
  ```

- This program will allow temperatures which are not integers

33

Program - Analysis

Converting from Fahrenheit to Celsius

- Defining `SCALE_FACTOR` to be `(5.0f/9.0f)` instead of `(5/9)` is important
- Note the use of `%.1f` to display celsius with just one digit after decimal

37

Identifiers

Keywords

- The following ***keywords*** can't be used as identifiers:

auto	enum	restrict*	unsigned
break	extern	return	void
case	float	short	volatile
char	for	signed	while
const	goto	sizeof	_Bool*
continue	if	static	_Complex*
default	inline*	struct	_Imaginary*
do	int	switch	
double	long	typedef	
else	register	union	